

B.SC.CSIT 4th Semester
Operating System: CSC259
LAB 05

TITLE

FAMILIARIZATION WITH BEST FIT AND FIRST FIT MEMORY MANAGEMENT.

OBJECTIVES

IMPLEMENTATION OF BEST FIT MEMORY ALLOCATION AND FIRST FIT MEMORY ALLOCATION IN LAB

REQUIRED THEORY

Memory management is the functionality of an operating system which handles or manages primary memory and moves processes back and forth between main memory and disk during execution. Memory management keeps track of each and every memory location, regardless of whether it is allocated to some process or it is free. It checks how much memory is to be allocated to processes. It decides which process will get memory at what time. It tracks whenever some memory gets freed or unallocated and correspondingly it updates the status.

First Fit: In the first fit approach is to allocate the first free partition or hole large enough which can accommodate the process. It finishes after finding the first suitable free partition

Best Fit: The best fit deals with allocating the smallest free partition which meets the requirement of the requesting process. This algorithm first searches the entire list of free partitions and considers the smallest hole that is adequate. It then tries to find a hole which is close to actual process size needed.

SOURCE CODE

1. BEST FIT MEMORY ALLOCATION

```
#include<stdio.h>

int main() {

int fragment[20],b[20],p[20],i,j,nb,np,temp,lowest=9999;

static int barray[20],parray[20];
```

```

printf("\n\t\t\tMemory Management Scheme - Best Fit");

printf("\nEnter the number of blocks:");

scanf("%d",&nb);

printf("Enter the number of processes:");

scanf("%d",&np);

printf("\nEnter the size of the blocks:-\n");

for(i=1;i<=nb;i++)

{

    printf("Block no.%d:",i);

    scanf("%d",&b[i]);

}

printf("\nEnter the size of the processes :-\n");

for(i=1;i<=np;i++)

{

    printf("Process no.%d:",i);

    scanf("%d",&p[i]);

}

for(i=1;i<=np;i++)

{

    for(j=1;j<=nb;j++)

    {

        if(barray[j]!=1)

        {

            temp=b[j]-p[i];

            if(temp>=0)

if(lowest>temp) {

```

```

                                parray[i]=j; lowest=temp;
                                }
                                }
                                }
fragment[i]=lowest;
                                barray[parray[i]]=1; lowest=10000;
                                }
                                printf("\nProcess_no\tProcess_size\tBlock_no\tBlock_size\tFragment");
                                for(i=1;i<=np && parray[i]!=0;i++)
                                printf("\n%d\t%d\t%d\t%d\t%d",i,p[i],parray[i],b[parray[i]],fragment[i]);
                                return 0;
                                }

```

2. FIRST FIT MEMORY ALLOCATION

In the First Fit Memory Management Scheme, we check the block in the sequential manner i.e we take the first process and compare its size with the first block. If the size is less than the size of the first block then only it is allocated. Otherwise, we move to the second block and this process is continuously going on until all processes are allocated.

ALGORITHM OF FIRST FIT:

- 1 Step: START.
- 2 Step: At first get the no of processes and blocks.
- 3 Step: Allocate the process by if(size of block>=size of the process) then allocate the process else move to the next block.
- 4 Step: Now Display the processes with blocks and allocate to respective process.
- 5 Step: STOP.

SOURCE CODE:

```

#include<stdio.h>

int main(){
    int bsize[10], psize[10], bno, pno, flags[10], allocation[10], i, j;

```

```

for(i = 0; i< 10; i++)
{  flags[i] = 0; allocation[i] = -1;  }

printf("Enter no. of blocks: ");

scanf("%d", &bno);

printf("\nEnter size of each block: ");

for(i = 0; i<bno; i++)

    scanf("%d", &bsize[i]);

printf("\nEnter no. of processes: ");

scanf("%d", &pno);

printf("\nEnter size of each process: ");

for(i = 0; i<pno; i++)

    scanf("%d", &psize[i]);

for(i = 0; i<pno; i++)    //allocation as per first fit

    for(j = 0; j <bno; j++)

        if(flags[j] == 0 &&bsize[j] >= psize[i])

        {

            allocation[j] = i;  flags[j] = 1;  break;

        }

//display allocation details

printf("\nBlock no.\tsize\t\tprocess no.\t\tsize");

for(i = 0; i<bno; i++)

{

    printf("\n%d\t\t%d\t\t", i+1, bsize[i]);

    if(flags[i] == 1)

        printf("%d\t\t\t%d",allocation[i]+1,psize[allocation[i]]);

    else  printf("Not allocated");  }

```

```
return 0;}
```

OUTPUT

1. Best Fit

```
ankit@ankit-virtual-machine:~$ ls
ankit      Desktop    Downloads  lab04  Pictures  snap      Videos
ankit.txt  Documents  lab02      Music  Public    Templates
ankit@ankit-virtual-machine:~$ mkdir lab05
ankit@ankit-virtual-machine:~$ cd lab05
ankit@ankit-virtual-machine:~/lab05$ touch bestfit.c
ankit@ankit-virtual-machine:~/lab05$ nano bestfit.c
ankit@ankit-virtual-machine:~/lab05$ ls
bestfit.c
ankit@ankit-virtual-machine:~/lab05$ gcc -o bestfit bestfit.c
ankit@ankit-virtual-machine:~/lab05$ ls
bestfit  bestfit.c
ankit@ankit-virtual-machine:~/lab05$ ./bestfit

                                Memory Management Scheme - Best Fit
Enter the number of blocks:8
Enter the number of processes:3

Enter the size of the blocks:-
Block no.1:14
Block no.2:17
Block no.3:8
Block no.4:9
Block no.5:14
Block no.6:5
Block no.7:6
Block no.8:3

Enter the size of the processes :-
Process no.1:5
Process no.2:8
Process no.3:7

Process_no      Process_size    Block_no      Block_size    Fragment
1                5                6                5                0
2                8                3                8                0
ankit@ankit-virtual-machine:~/lab05$
```

2. First Fit

```
ankit@ankit-virtual-machine:~$ ls
ankit      Desktop  Downloads  lab04  Music    Public  Templates
ankit.txt  Documents  lab02      lab05  Pictures  snap    Videos
ankit@ankit-virtual-machine:~$ cd lab05
ankit@ankit-virtual-machine:~/lab05$ touch firstfit.c
ankit@ankit-virtual-machine:~/lab05$ nano firstfit.c
ankit@ankit-virtual-machine:~/lab05$ ls
bestfit  bestfit.c  firstfit.c
ankit@ankit-virtual-machine:~/lab05$ gcc -o firstfit firstfit.c
ankit@ankit-virtual-machine:~/lab05$ ls
bestfit  bestfit.c  firstfit  firstfit.c
ankit@ankit-virtual-machine:~/lab05$ ./firstfit
Enter no. of blocks: 3

Enter size of each block: 8
9
10

Enter no. of processes: 3

Enter size of each process: 40
50
60

Block no.      size      process no.      size
1              8        Not allocated
2              9        Not allocated
ankit@ankit-virtual-machine:~/lab05$ ./firstfit
Enter no. of blocks: 3

Enter size of each block: 8
9
10

Enter no. of processes: 3

Enter size of each process: 10 11 12

Block no.      size      process no.      size
1              8        Not allocated
2              9        Not allocated
3              10       1               10ankit@ankit-virtual-machine:~/lab05$
```

CONCLUSION

HENCE WE STUDIED AND IMPLEMENTED BEST FIT AND FIRST FIT MEMORY ALLOCATION IN UBUNTU OPEARTING SYSTEM